

FireGuard IoT — Trabalho 2

Autores: Richardson, Wallace, Emanuele, Vinícius

1. Introdução

O projeto **FireGuard IoT** consiste em um sistema de alarme de incêndio inteligente que integra hardware (Arduino) e software (Backend em Python e Frontend Web). O sistema monitora continuamente o ambiente através de um sensor de chama infravermelho e fornece feedback visual e sonoro local, além de transmitir os dados para um dashboard web em tempo real e armazená-los em um banco de dados para consulta histórica.

Este documento apresenta a lista de requisitos, o diagrama de casos de uso e o código-fonte completo do sistema, contemplando todos os critérios estabelecidos para o Trabalho 2.

2. Requisitos do Sistema

2.1 Requisitos Funcionais (RF)

ID	Requisito	Descrição
RF-01	Leitura do Sensor	O sistema deve ler continuamente os valores analógicos (0 a 1023) do sensor de chama.
RF-02	Classificação de Estado	O sistema deve classificar a leitura em: SEGURO (>700), ALERTA (301-700) e PERIGO (<=300).
RF-03	Feedback Visual Local	O hardware deve acender LEDs correspondentes ao estado atual (Verde, Amarelo, Vermelho).
RF-04	Feedback Sonoro Local	O hardware deve ativar um alarme sonoro (buzzer) contínuo no estado PERIGO.
RF-05	Silenciamento Temporário	O usuário deve poder silenciar o alarme sonoro temporariamente (10s) via botão físico.
RF-06	Contagem Regressiva	Durante o silenciamento, exibir contagem regressiva em um display de 7 segmentos.
RF-07	Transmissão de Dados	Transmitir os dados do sensor e estado via porta serial USB para o computador.
RF-08	Armazenamento (Backend)	O backend deve receber os dados e armazená-los em um banco de dados SQLite.
RF-09	Dashboard em Tempo Real	O frontend deve exibir um dashboard com os dados do sensor, estado e gráfico em tempo real.
RF-10	Consulta de Histórico	O frontend deve possuir uma tela para consulta dos dados históricos armazenados.
RF-11	Filtros de Consulta	Permitir filtrar o histórico por data, intervalo de datas e estado.
RF-12	Estatísticas Resumidas	Exibir resumo dos dados filtrados (total, quantidades por estado, min/max/média).

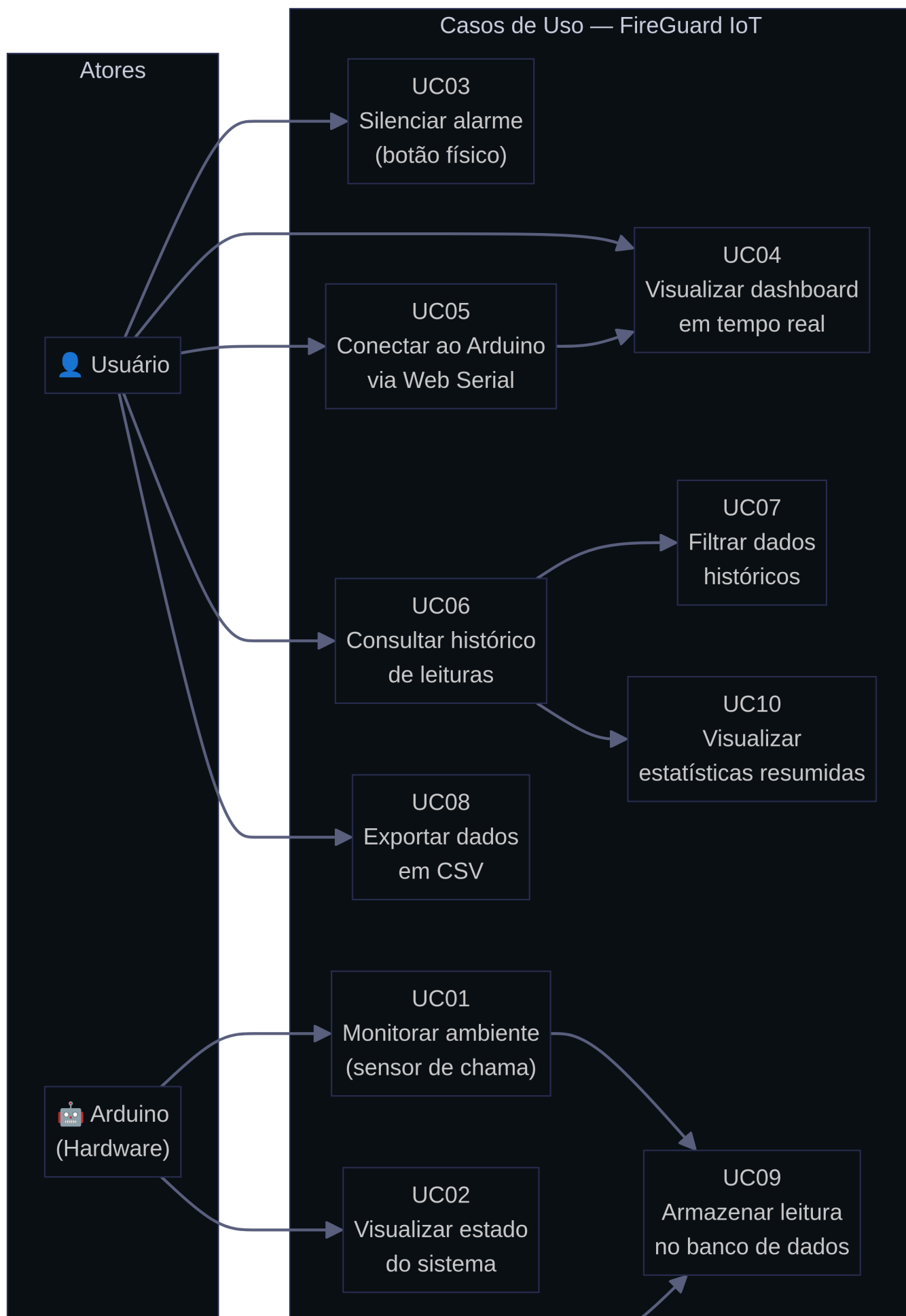
ID	Requisito	Descrição
RF-13	Exportação de Dados	O usuário deve poder exportar os dados do histórico para um arquivo CSV.

2.2 Requisitos Não Funcionais (RNF)

ID	Requisito	Categoria	Descrição
RNF-01	Desempenho (Leitura)	Eficiência	O sensor deve ser lido em intervalos não superiores a 150 ms.
RNF-02	Desempenho (Resposta)	Eficiência	A transição de estado local deve ocorrer em até 200 ms após a detecção.
RNF-03	Confiabilidade (Histerese)	Confiabilidade	Deve existir histerese de 30 unidades nos limiares para evitar oscilação.
RNF-04	Segurança (Alarme)	Segurança	O alarme sonoro no estado PERIGO deve operar a 2500 Hz.
RNF-05	Usabilidade (Interface)	Usabilidade	A interface gráfica deve ser responsiva (mobile e desktop).
RNF-06	Portabilidade (Software)	Portabilidade	O Backend deve ser em Python (Flask) e o banco de dados embutido (SQLite).
RNF-07	Portabilidade (Navegador)	Portabilidade	O Dashboard deve funcionar via Web Serial API em navegadores baseados em Chromium.

3. Diagrama de Casos de Uso

Abaixo está o diagrama de casos de uso que ilustra as interações entre os atores (Usuário, Arduino e Backend) e as funcionalidades do sistema.





Sistema
(Backend)

4. Código-Fonte

4.1 Código Embarcado (Arduino C++)

Arquivo: `codigo.ino`

```

// =====
// Alarme de Incêndio IoT – Firmware Arduino (Trabalho 1 e 2)
// =====

#include <Arduino.h>

// — Pinos —————
const int PINO_SENSOR = A0;
const int PINO_BOTAO = 2;
const int PINO_BUZZER = 3;

// LEDs
const int PINO_LED_G1 = 13;
const int PINO_LED_G2 = 12;
const int PINO_LED_Y1 = 11;
const int PINO_LED_Y2 = 10;
const int PINO_LED_R1 = 9;
const int PINO_LED_R2 = 8;

// Display 7 segmentos
const int PINO_SEG_A = 4;
const int PINO_SEG_B = 5;
const int PINO_SEG_C = 6;
const int PINO_SEG_D = 7;
const int PINO_SEG_E = A1;
const int PINO_SEG_F = A2;
const int PINO_SEG_G = A3;

// — Constantes e Limiares —————
const int LIMIAR_SEGURO = 700;
const int LIMIAR_ALERTA = 300;
const int HISTERESE = 30;

const unsigned long INTERVALO_LEITURA_MS = 100;
const int DEBOUNCE_MS = 50;
const int FREQ_ALARME_HZ = 2500;
const int CONTAGEM_REGRESSIVA = 10;

// — Estados —————
enum EstadoSistema {
    SEGURO,
    ALERTA,
    PERIGO,

```

```

    SILENCIADO
};

EstadoSistema estadoAtual = SEGURO;
int nivelSensor = 1023;
unsigned long ultimaLeituraTempo = 0;
bool botaoPressionadoAnterior = false;
unsigned long ultimoDebounceTempo = 0;

// — Tabela de Segmentos (Cátodo Comum) —
const byte DIGITOS[10][7] = {
    {1, 1, 1, 1, 1, 1, 0}, // 0
    {0, 1, 1, 0, 0, 0, 0}, // 1
    {1, 1, 0, 1, 1, 0, 1}, // 2
    {1, 1, 1, 1, 0, 0, 1}, // 3
    {0, 1, 1, 0, 0, 1, 1}, // 4
    {1, 0, 1, 1, 0, 1, 1}, // 5
    {1, 0, 1, 1, 1, 1, 1}, // 6
    {1, 1, 1, 0, 0, 0, 0}, // 7
    {1, 1, 1, 1, 1, 1, 1}, // 8
    {1, 1, 1, 1, 0, 1, 1}  // 9
};

// — Protótipos —
void configurarPinos();
void lerSensor();
EstadoSistema calcularEstado(int valor);
void aplicarEstado(EstadoSistema novoEstado);
void verificarBotao();
void iniciarSilenciamento();
void mostrarNumero(int numero);
void desligarDisplay();
void imprimirEstado();

// — Setup —
void setup() {
    Serial.begin(9600);
    configurarPinos();

    Serial.println(F("≡ FireGuard IoT Inicializado ≡"));
    Serial.print(F("Limiares - Seguro: >")); Serial.print(LIMIAR_SEGURO);
    Serial.print(F(" | Alerta: ")); Serial.print(LIMIAR_ALERTA);
    Serial.print(F("-")); Serial.print(LIMIAR_SEGURO);
    Serial.print(F(" | Perigo: <")); Serial.println(LIMIAR_ALERTA);
}

```



```
    aplicarEstado(SEGURO);
}

// — Loop —
void loop() {
    unsigned long tempoAtual = millis();

    if (tempoAtual - ultimaLeituraTempo ≥ INTERVALO_LEITURA_MS) {
        lerSensor();
        ultimaLeituraTempo = tempoAtual;
    }

    verificarBotao();
}

// — Funções —
void configurarPinos() {
    pinMode(PINO_BOTAO, INPUT_PULLUP);
    pinMode(PINO_BUZZER, OUTPUT);

    int pinosSaida[] = {
        PINO_LED_G1, PINO_LED_G2, PINO_LED_Y1, PINO_LED_Y2, PINO_LED_R1, PINO.
        PINO_SEG_A, PINO_SEG_B, PINO_SEG_C, PINO_SEG_D, PINO_SEG_E, PINO_SEG_I
    };

    for (int pino : pinosSaida) {
        pinMode(pino, OUTPUT);
        digitalWrite(pino, LOW);
    }
}

void lerSensor() {
    nivelSensor = analogRead(PINO_SENSOR);

    if (estadoAtual ≠ SILENCIADO) {
        EstadoSistema novoEstado = calcularEstado(nivelSensor);
        if (novoEstado ≠ estadoAtual) {
            aplicarEstado(novoEstado);
        } else {
            imprimirEstado();
        }
    } else {
        imprimirEstado();
    }
}
```

```
}  
}
```

```
EstadoSistema calcularEstado(int valor) {  
    if (estadoAtual == SEGURO && valor ≤ LIMIAR_SEGURO - HISTERESE) {  
        return (valor ≤ LIMIAR_ALERTA) ? PERIGO : ALERTA;  
    } else if (estadoAtual == ALERTA) {  
        if (valor > LIMIAR_SEGURO + HISTERESE) return SEGURO;  
        if (valor ≤ LIMIAR_ALERTA - HISTERESE) return PERIGO;  
    } else if (estadoAtual == PERIGO && valor > LIMIAR_ALERTA + HISTERESE) {  
        return (valor > LIMIAR_SEGURO) ? SEGURO : ALERTA;  
    }  
    return estadoAtual;  
}
```

```
void aplicarEstado(EstadoSistema novoEstado) {  
    estadoAtual = novoEstado;  
  
    digitalWrite(PINO_LED_G1, novoEstado == SEGURO);  
    digitalWrite(PINO_LED_G2, novoEstado == SEGURO);  
    digitalWrite(PINO_LED_Y1, novoEstado == ALERTA);  
    digitalWrite(PINO_LED_Y2, novoEstado == ALERTA);  
    digitalWrite(PINO_LED_R1, novoEstado == PERIGO);  
    digitalWrite(PINO_LED_R2, novoEstado == PERIGO);  
  
    if (novoEstado == PERIGO) {  
        tone(PINO_BUZZER, FREQ_ALARME_HZ);  
    } else {  
        noTone(PINO_BUZZER);  
    }  
  
    imprimirEstado();  
}
```

```
void verificarBotao() {  
    int leituraBotao = digitalRead(PINO_BOTAO);  
    unsigned long tempoAtual = millis();  
  
    if (leituraBotao ≠ botaoPressionadoAnterior) {  
        ultimoDebounceTempo = tempoAtual;  
    }  
  
    if ((tempoAtual - ultimoDebounceTempo) > DEBOUNCE_MS) {  
        if (leituraBotao == LOW && estadoAtual == PERIGO) {
```

```

        iniciarSilenciamento();
    }
}

botaoPressionadoAnterior = leituraBotao;
}

void iniciarSilenciamento() {
    estadoAtual = SILENCIADO;
    noTone(PINO_BUZZER);
    Serial.println(F("Alarme SILENCIADO pelo usuario.));

    for (int i = CONTAGEM_REGRESSIVA - 1; i ≥ 0; i--) {
        mostrarNumero(i);
        Serial.print(F("Silenciado, reiniciando em "));
        Serial.print(i);
        Serial.println(F("s..."));
        delay(1000);
    }

    desligarDisplay();
    Serial.println(F("Monitoramento retomado.));
    aplicarEstado(calcularEstado(analogRead(PINO_SENSOR)));
}

void mostrarNumero(int num) {
    if (num < 0 || num > 9) return;
    digitalWrite(PINO_SEG_A, DIGITOS[num][0]);
    digitalWrite(PINO_SEG_B, DIGITOS[num][1]);
    digitalWrite(PINO_SEG_C, DIGITOS[num][2]);
    digitalWrite(PINO_SEG_D, DIGITOS[num][3]);
    digitalWrite(PINO_SEG_E, DIGITOS[num][4]);
    digitalWrite(PINO_SEG_F, DIGITOS[num][5]);
    digitalWrite(PINO_SEG_G, DIGITOS[num][6]);
}

void desligarDisplay() {
    int pinos[] = {PINO_SEG_A, PINO_SEG_B, PINO_SEG_C, PINO_SEG_D, PINO_SEG_E, PINO_SEG_F, PINO_SEG_G};
    for (int pino : pinos) digitalWrite(pino, LOW);
}

void imprimirEstado() {
    Serial.print(F("Sensor: "));
    Serial.print(nivelSensor);
}

```

```
Serial.print(F(" | Estado: "));

switch (estadoAtual) {
  case SEGURO:      Serial.println(F("SEGURO (verde)")); break;
  case ALERTA:      Serial.println(F("ALERTA (amarelo)")); break;
  case PERIGO:      Serial.println(F("PERIGO (vermelho) - BUZZER ON")); break;
  case SILENCIADO:  Serial.println(F("SILENCIADO")); break;
}
}
```

4.2 Backend em Python (Flask + SQLite)

Arquivo: backend/server.py

```

import sqlite3
import threading
import time
import re
import os

from datetime import datetime, timedelta
from flask import Flask, jsonify, request, send_from_directory
from flask_cors import CORS

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.path.join(BASE_DIR, "fireguard.db")
FRONTEND = os.path.join(BASE_DIR, "..", "frontend")
SERIAL_PORT = os.environ.get("SERIAL_PORT", "")
BAUD_RATE = 9600

app = Flask(__name__, static_folder=FRONTEND)
CORS(app)

def get_db():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    with get_db() as conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS leituras (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                timestamp TEXT NOT NULL,
                sensor INTEGER NOT NULL,
                estado TEXT NOT NULL
            )
        """)
        conn.commit()

def salvar_leitura(sensor: int, estado: str):
    ts = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with get_db() as conn:
        conn.execute("INSERT INTO leituras (timestamp, sensor, estado) VALUES (?, ?, ?)")
        conn.commit()

def ler_serial():
    try:

```

```

import serial
ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=2)
while True:
    linha = ser.readline().decode("utf-8", errors="ignore").strip
    if not linha: continue
    m_sensor = re.search(r"Sensor:\s*(\d+)", linha, re.IGNORECASE)
    m_estado = re.search(r"Estado:\s*(\w+)", linha, re.IGNORECASE)
    if m_sensor and m_estado:
        salvar_leitura(int(m_sensor.group(1)), m_estado.group(1))
except Exception:
    simular_leituras()

def simular_leituras():
    import random
    estados = ["SEGURO"]*20 + ["ALERTA"]*10 + ["PERIGO"]*8 + ["SEGURO"]*10
    idx = 0
    while True:
        estado = estados[idx % len(estados)]
        sensor = random.randint(400, 700) if estado != "PERIGO" else random.randint(800, 1000)
        salvar_leitura(sensor, estado)
        idx += 1
        time.sleep(2)

@app.route("/api/leituras", methods=["GET"])
def get_leituras():
    conn = get_db()
    estado = request.args.get("estado", "").upper()
    limite = int(request.args.get("limite", 200))

    where = "WHERE estado = ?" if estado and estado != "TODOS" else ""
    params = [estado, limite] if where else [limite]

    rows = conn.execute(f"SELECT * FROM leituras {where} ORDER BY id DESC")
    conn.close()
    return jsonify([dict(r) for r in rows])

@app.route("/api/leituras/resumo", methods=["GET"])
def get_resumo():
    conn = get_db()
    stats = conn.execute("""
        SELECT COUNT(*) as total, MIN(sensor) as min, MAX(sensor) as max,
        SUM(CASE WHEN estado='SEGURO' THEN 1 ELSE 0 END) as seguro,
        SUM(CASE WHEN estado='PERIGO' THEN 1 ELSE 0 END) as perigo
        FROM leituras
    """)
    return jsonify(stats.fetchone())

```

```

        """).fetchone()
        conn.close()
        return jsonify(dict(stats))

@app.route("/")
def index():
    return send_from_directory(FRONTEND, "historico.html")

@app.route("/<path:filename>")
def static_files(filename):
    return send_from_directory(FRONTEND, filename)

if __name__ == "__main__":
    init_db()
    threading.Thread(target=ler_serial, daemon=True).start()
    app.run(host="0.0.0.0", port=5000)

```

(O código HTML/JS/CSS completo do frontend encontra-se na pasta `/frontend` fornecida no arquivo ZIP anexo).